tantor

PG BootCamp Russia 2026 Moscow
PGBootCamp.ru

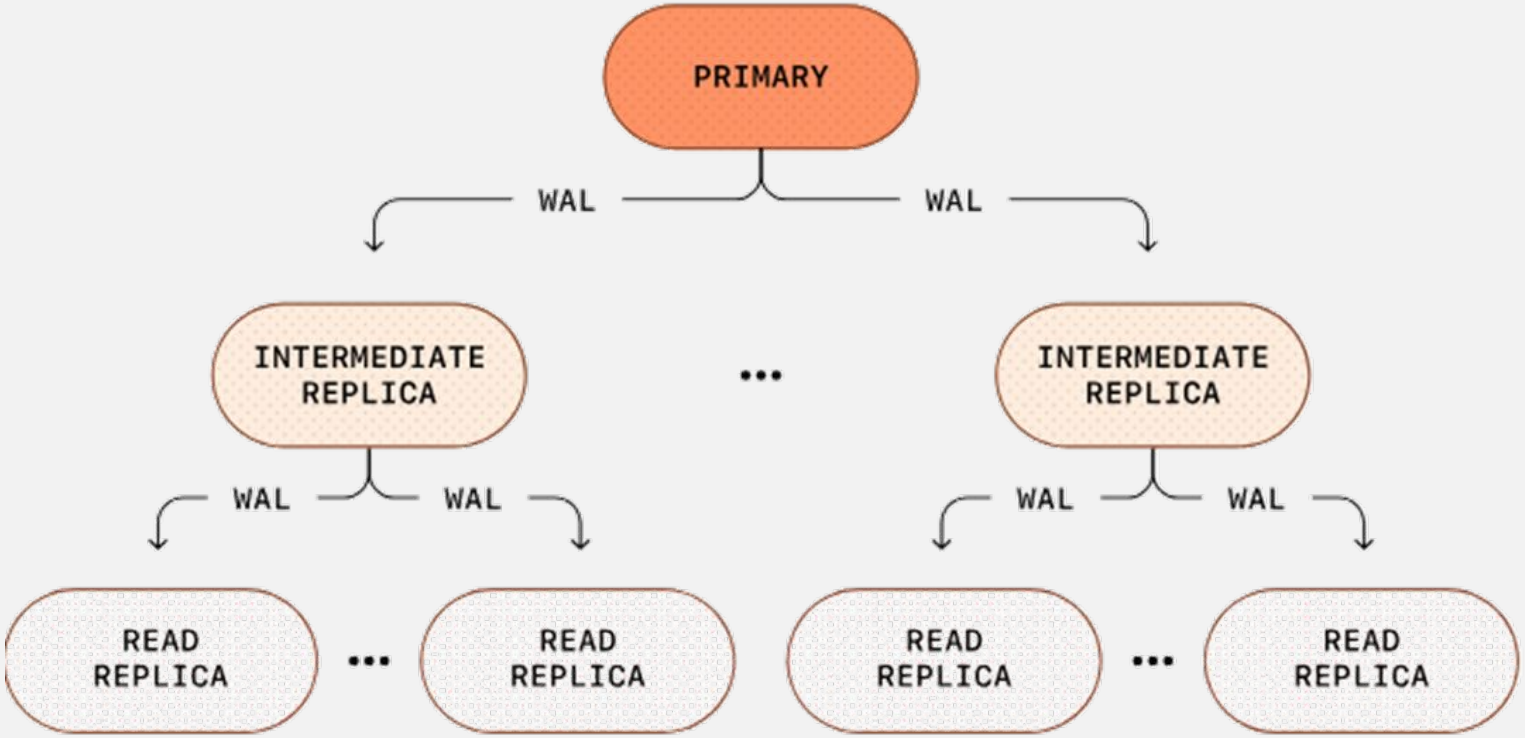
Опыт вынесения OLAP-нагрузки на реплику

Илья Рожнев, разработчик «Тантор Лабс»
Александр Симонов, руководитель направления 1С «Тантор Лабс»

Потребности высоконагруженных систем

1 мастер и 50 читающих реплик

OpenAI



https://openai.com/ru-RU/index/scaling-postgresql/

Потребности высоконагруженных систем

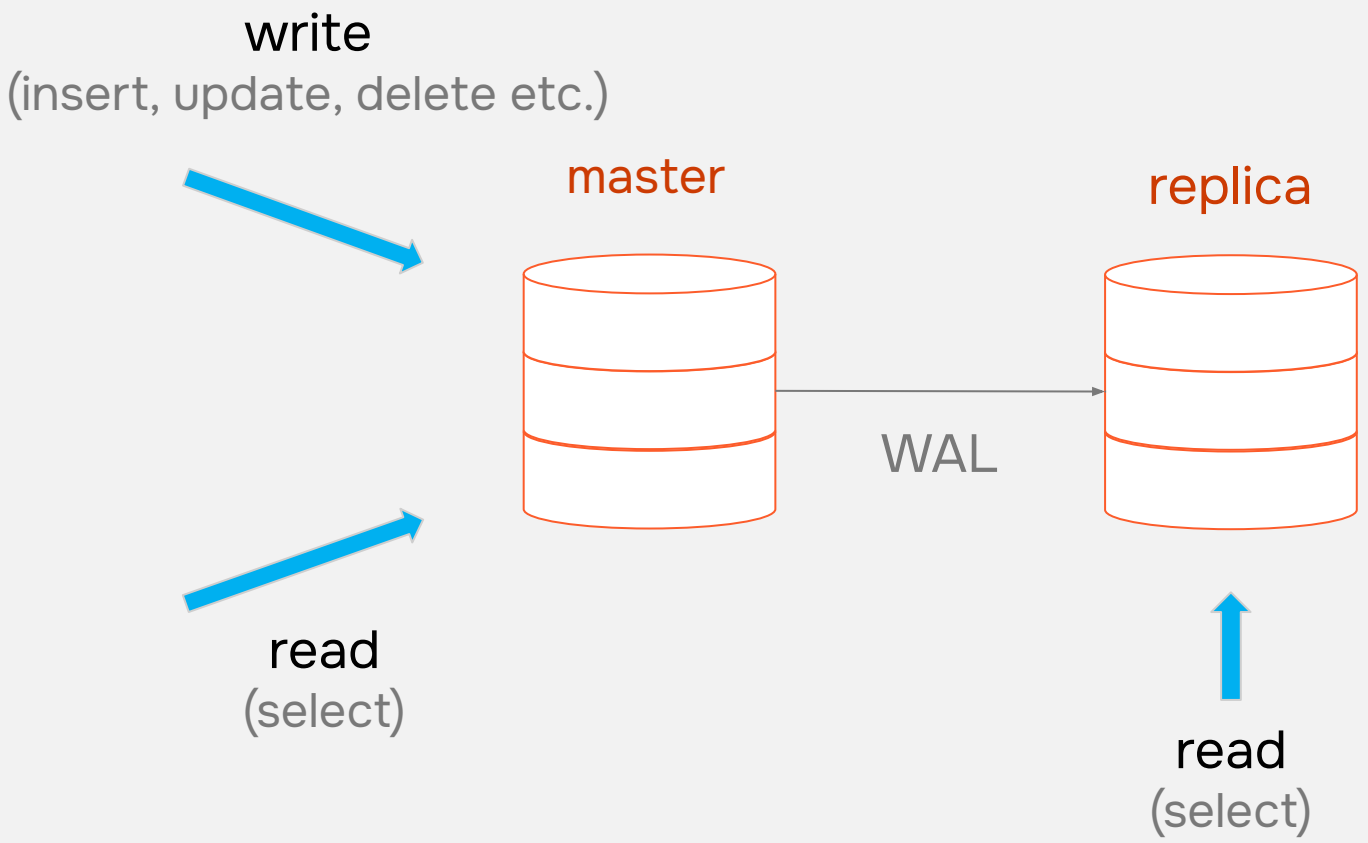
Особенности:

1. Вычислительные ресурсы реплик простаивают
2. Аналитические запросы создают основную нагрузку

Хорошо бы вынести часть нагрузки на реплику!

Как работает HOT STANDBY

- HOT-STANDBY используется как дорогой, живой бекап
- Работают только SELECT-запросы
- Реплика READ-ONLY



Создание временной таблицы на реплике

Проблема

```
create temporary table tt1 (_Q_001_F_000RRef bytea)
without oids;
>
ERROR: cannot execute CREATE TABLE in a read-only
transaction SQL state: 25006
```

Создание временной таблицы на реплике

Почему временные таблицы запрещено создавать на реплике?

Создание временной таблицы на реплике

Почему временные таблицы запрещено создавать на реплике?

SQL

```
CREATE TEMPORARY TABLE
tt1
(_q_001_f_000rref
 bytea)
```

INSERT **pg_class**
Все объекты БД и их OID
→ запись о таблице "tt1"

INSERT **pg_attribute**
Поля таблиц
→ "_q_001_f_000rref" в tt1

INSERT **pg_type**
Типы данных полей
→ "_q_001_f_000rref" : bytea

INSERT **pg_index**
Индексы таблиц
→ индекс "tmpind_0"

INSERT **pg_depend**
Зависимости объектов БД
→ "tmpind_0" зависит от tt1

Создание временной таблицы на реплике

Почему временные таблицы запрещено создавать на реплике?

SQL

```
CREATE TEMPORARY TABLE
tt1
(_q_001_f_000rref
 bytea)
```

INSERT **pg_class**
Все объекты БД и их OID
→ запись о таблице "tt1"

INSERT **pg_attribute**
Поля таблиц
→ "_q_001_f_000rref" в tt1

INSERT **pg_type**
Типы данных полей
→ "_q_001_f_000rref" : bytea

INSERT **pg_index**
Индексы таблиц
→ индекс "tmpind_0"

INSERT **pg_depend**
Зависимости объектов БД
→ "tmpind_0" зависит от tt1

Проблемы:

1. Расхождение данных
2. Конфликты WAL-записей
3. Коллизии OID

Что если не таблицы?

Мы можем:

CTE

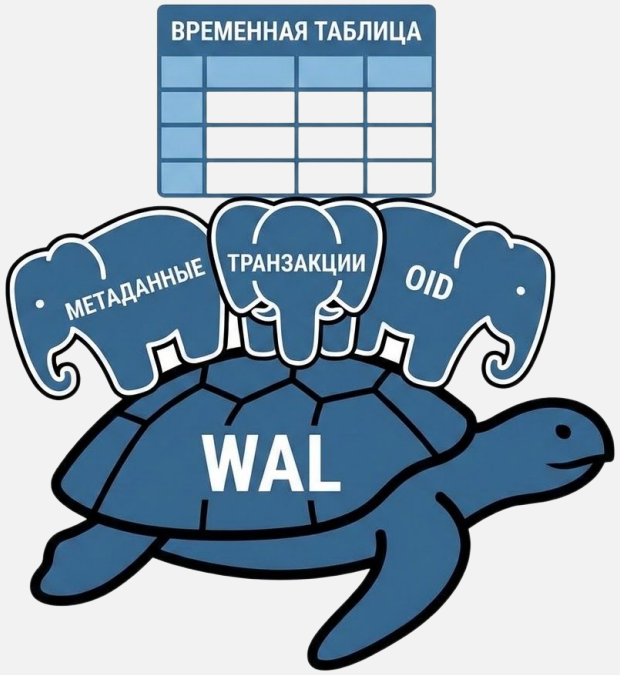
```
WITH ... AS
...
```

Обработка на стороне приложений

Вернуться на мастер

Как научить слона плавать?

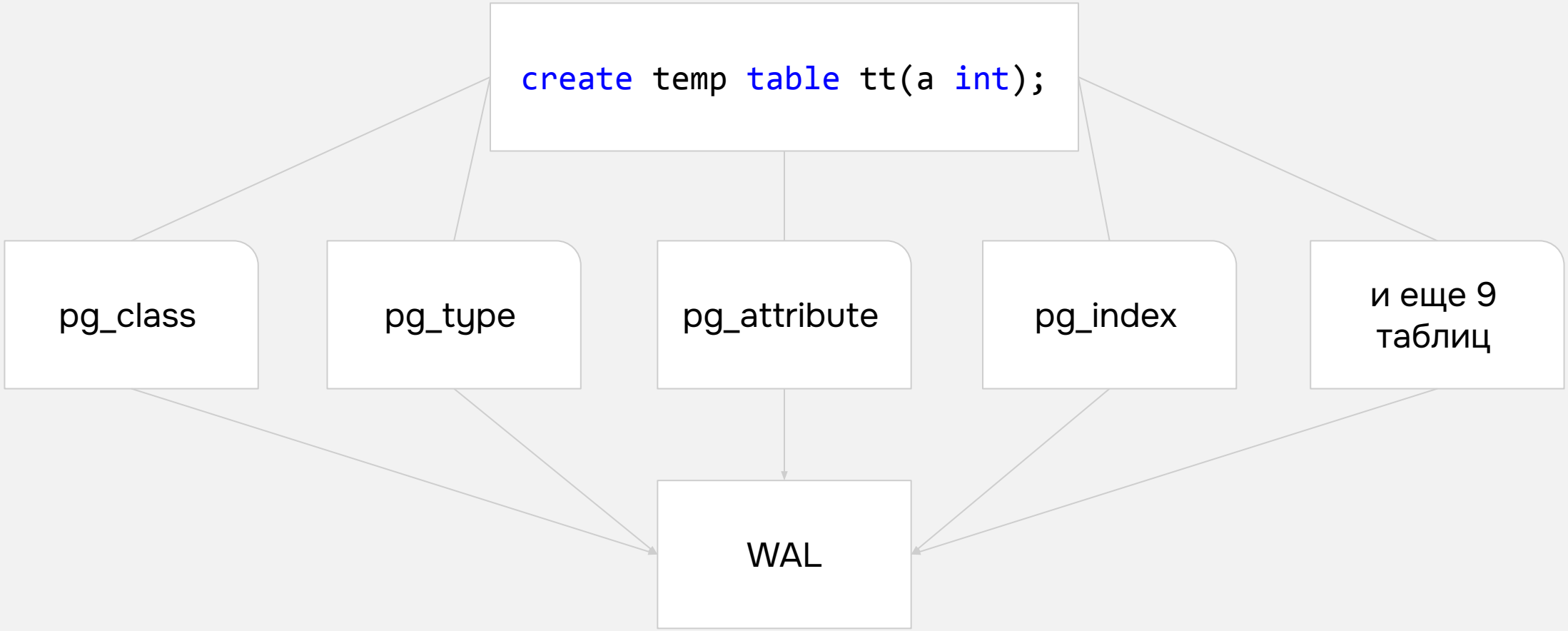
- Убрать записи в системные каталоги
- Убрать записи в pg_xact и WAL-записи с транзакциями
- Убрать WAL-записи от OID



Часть 1.

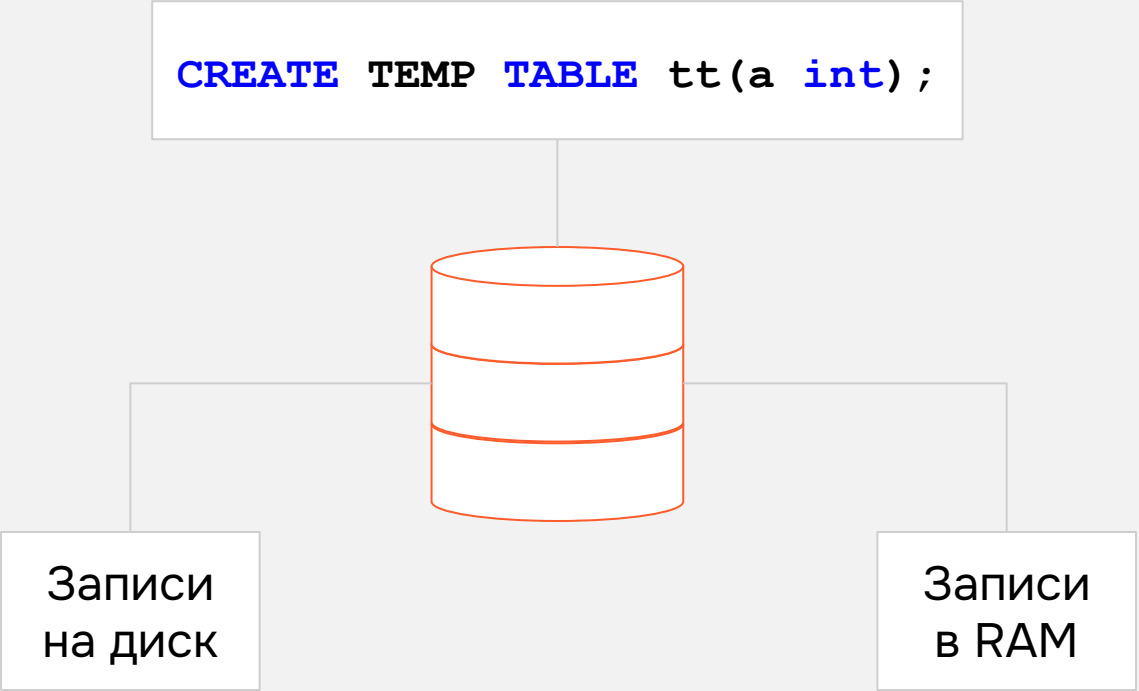
Записи в системный каталог

Метаданные таблицы идут в общие каталоги



Перехватить записи в системный каталог

Решение: не дать данным дойти до диска и обмануть ядро Postgres, что данные там



Где будем перехватывать?

```
/* Лучше использовать CatalogTupleInsert */
void
heap_insert(Relation relation, HeapTuple tup, CommandId cid,
            int options, BulkInsertState bistate)
{
      /* ... */
    if (inmem_insert(relation, tup))
        return;
    /* ... */
}

bool
inmem_insert(Relation relation, HeapTuple tup)
{
    if (!is_inmem_tuple(relation, tup))
        return false;
}
```

Как понять, что нужно перехватить кортеж?

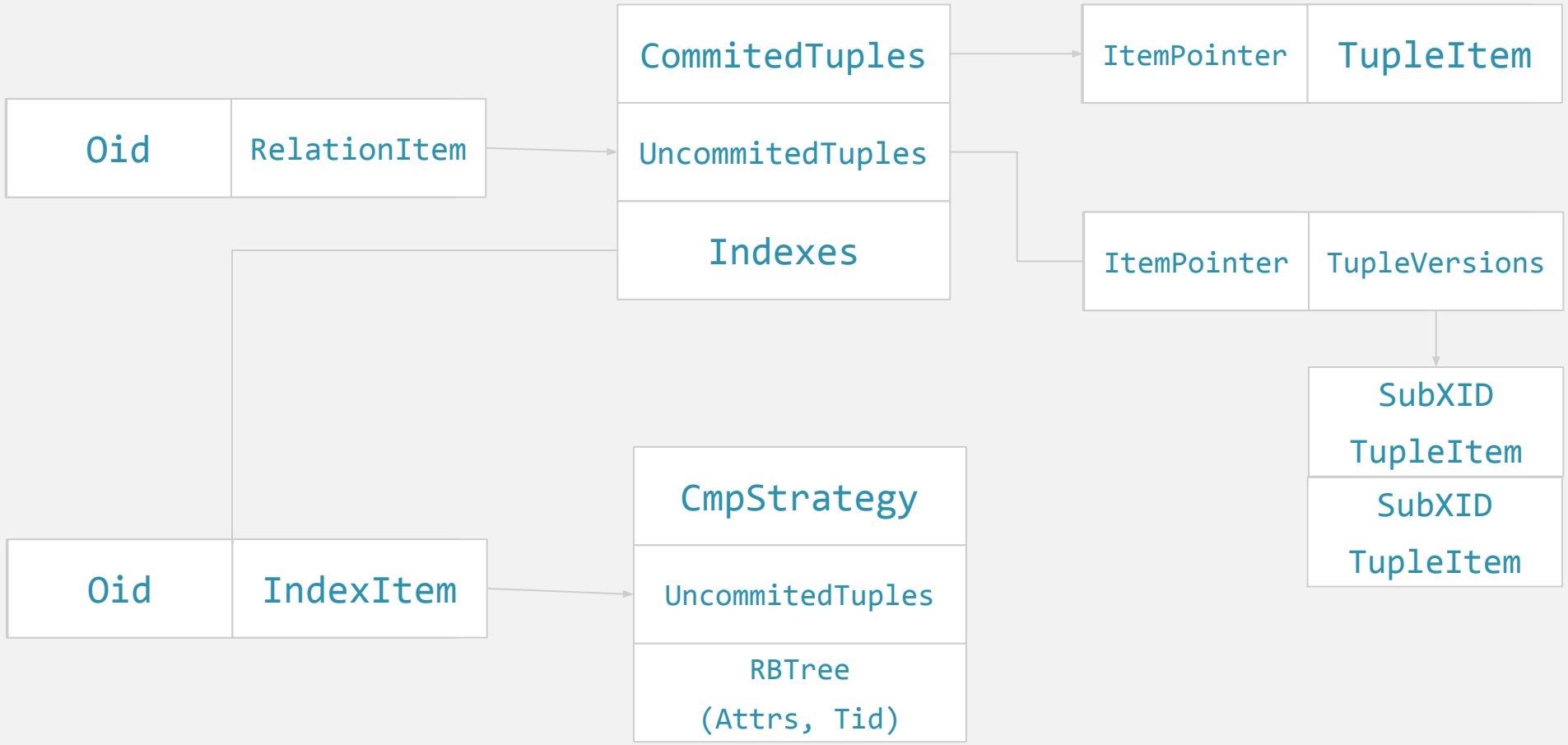
Мы использовали прекрасное решение от 1С:

```
#define BEGIN_TEMP_TABLE_SCOPE(level) \
    _save_state = temp_table_scope; \
    if (sigsetjmp(_buf, 0) == 0) {    /* Точка отката при ошибке */ \
        PG_exception_stack = &_buf;   \
        temp_table_scope = level;     /* Входим в Scope */ \
    } else { _throw = true; }         \
    if (!_throw) {

#define END_TEMP_TABLE_SCOPE() \
    } \
    temp_table_scope = _save_state;   /* Гарантированный возврат
состояния */ \
    if (_throw) PG_RE_THROW();        /* Пробрасываем ошибку выше */

/* Пример использования */
BEGIN_TEMP_TABLE_SCOPE(relpersistence == RELPERSISTENCE_TEMP);
heap_create_with_catalog();
END_TEMP_TABLE_SCOPE();
```

Как хранить данные?



В итоге получаем:

```
bool
inmem_insert(Relation relation, HeapTuple tup)
{
    if (!IsTempTableScope())
        return false;
    TupleItem* tupitem = alloc_tuple_item(tup);
    heaptup->t_self = tupitem->tuple->t_self; // Тут кастомный
Tid
    /* Вставляем в наш RelationItem и в Indexes */
    insert_tuple_item(relation, tupitem);
    return true;
}
```

Кастомный ItemPointerData

```
static uint32 CurrentInmemBlockId = 0;
static uint16 CurrentInmemOffset = 1; /* 0 is considered invalid */
#define INMEM_ITEM_POINTER_BIT 0x0800
static ItemPointerData
GenInmemItemPointerData()
{
    ItemPointerData res;

    BlockIdSet(&(res.ip_blkid), CurrentInmemBlockId);
    res.ip_posid = CurrentInmemOffset | INMEM_ITEM_POINTER_BIT;

    CurrentInmemOffset++;

    if (CurrentInmemOffset > MaxHeapTuplesPerPage)
    {
        CurrentInmemOffset = 1;
        CurrentInmemBlockId++;
    }

    return res;
}
```

Также нужно реализовать update и delete

```
bool
inmem_update(Relation relation, ItemPointer otid, HeapTuple
newtup)
{
    if (!IsInmemItemPointer(otid))
        return false;

    TupleItem*  oldTupleEntry = find_tuple_item(otid, rel);
    TupleItem* newTupleEntry = alloc_tuple_item(newtup, rel);

    CacheInvalidateHeapTuple(relation, oldTupleEntry->tuple,
newTupleEntry->tuple);

    delete_tuple_item(otid, rel);
    insert_tuple_item(newTupleEntry, rel);
    return true;
}
```

```
bool
inmem_delete(Relation relation, ItemPointer tid)
{
    if (!IsInmemItemPointer(otid))
        return false;
    TupleItem* tup = find_tuple_item(otid, rel);
    CacheInvalidateHeapTuple(relation, tup->tuple, NULL);
    delete_tuple_item(otid, rel);
    return true;
}
```

Чтение: beginscan, endscan и getnext

```
void
inmem_beginscan(IndexScanDesc scan)
{
    if (!IsInmemIndex(scan->indexRelation))
        return;
    scan->inmem_rbdata = rbt_create();
}

void
inmem_endscan(IndexScanDesc scan)
{
    if (!IsInmemIndex(scan->indexRelation))
        return;
    pfree(scan->inmem_rbdata);
}
```

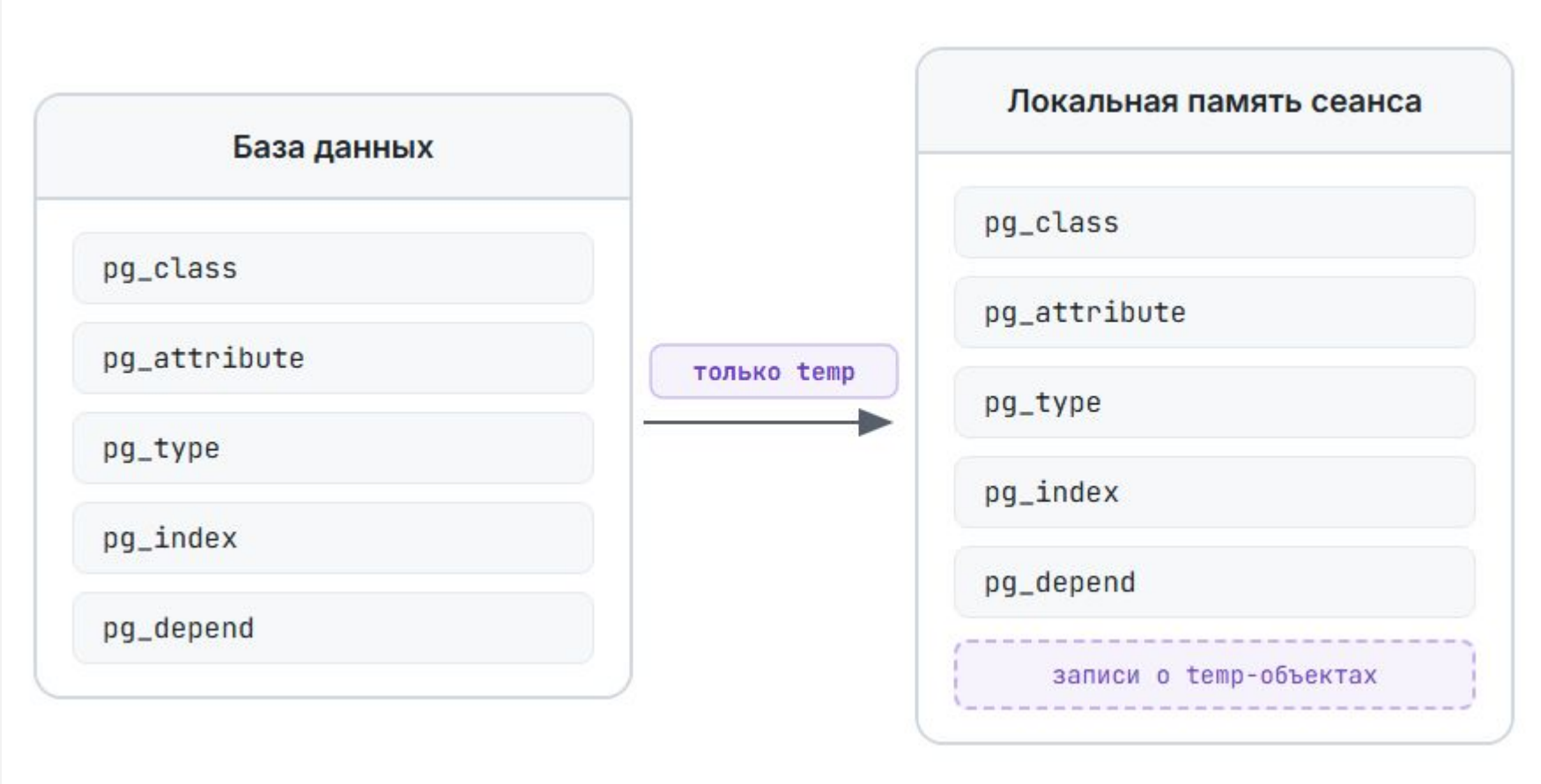
```
bool
inmem_getnext(IndexScanDesc scan, IndexScanDesc direction)
{
    if (!IsInmemIndex(scan->indexRelation))
        return false;
    if (!scan->inmem_inited)
         inmem_index_init_scan(scan);
    if (scan->inmem_size == 0)
        return scan->indexRelation->rd_indam->amgettuple(scan,
direction);



    scan->xs_itup = inmem_index_form_tuple(&inode->key, scan);
    rbt_delete((RBTree*)scan->inmem_rbdata, (RBTNode*)inode);

    return true;
}
```

Мы победили! Убрали метаданные



Часть 1.5

Прорубаем дорогу на реплике

Операции на реплике заблокированы. Разблокируем их!

```
standard_ProcessUtility()
{
/* ... */
    enableTempTableReplica =
IsTempTableOnReplicaEnabled() &&
ReplicaStmtAllowed(parsetree);
/* ... */
}
```

```
T_CreateSeqStmt
T_AlterSeqStmt
T_CreateTableAsStmt
T_CreateStmt
T_IndexStmt
T_DropStmt
T_TruncateStmt
T_AlterTableStmt
T_RenameStmt
T_CopyStmt
...
T_AlterExtensionStmt
```

Часть 2.

Записи XID

Небольшой хак

- Если транзакции не нужны – можно использовать Frozen Transaction Id

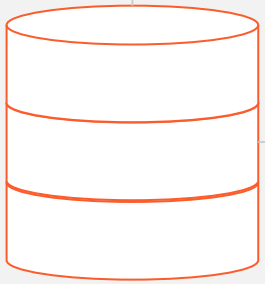
```
#define FrozenTransactionId        UINT64CONST(2)
```

- Минус в том, что мы теряем возможность использовать ROLLBACK/COMMIT

Снова перехватить стандартную логику!

Решение: не дать данным дойти до WAL

```
CREATE TEMP TABLE tt(a int);
```



Записи в RAM

Добавим счетчик транзакций

```
TransactionId NextReplicaLocalTransactionId =
FirstNormalTransactionId;
```

```
/*
 * Replica Local Transaction
Status
*/
typedef enum {
    RLX_INVALID = -1,
    RLX_INPROGRESS,
    RLX_COMMITTED,
    RLX_ABORTED
} RLXStatus;
```

TransactionId	RLXStatus
TransactionId	RLXStatus
TransactionId	RLXStatus

Логика MVCC

```
bool
HeapTupleSatisfiesReplicaLocal(HeapTuple htup, Snapshot snapshot)
{
    /* Check inserting transaction */
    switch (GetReplicaLocalTransactionStatus(xmin))
    {
        case RLX_ABORTED:
            return false;
        case RLX_COMMITTED:
            break;
       /* ... */
     }
    /* Check deleting transaction, if any */
    if (TransactionIdIsReplicaLocal(xmax))
    {
        switch (GetReplicaLocalTransactionStatus(xmax))
        {
            case RLX_COMMITTED:
                return false;
            case RLX_ABORTED:
                return true;
            /* ... */
        }
    }
```

Окно транзакций

```
bool
TransactionIdIsReplicaLocal(TransactionId xid)
{
   /* fast path */
    if (CheckXidInBitmap(xid))
    {
        xid_tracker->bitmap_hits++;
        return true;
    }
        xid_tracker->misses++;
        return false;

    /* Slow path: check hash table for older XIDs */
    hash_search(xid_tracker->older_xids_hash,
                                    &xid, HASH_FIND, &found);
}
```

Часть 3.

Генерация OID

Генерация OID крайне проста

```
Oid
GetNewReplicaLocalObjectId(void)
{

    static Oid new_oid = FirstNormalObjectId;
    return new_oid++;
}
```

Часть 4.

Нагрузочное тестирование

Механизм копий баз данных 1С

Возможности

- Использование любого количества реплик
- Вынос конкретных отчетов на реплики
- Выполнение отчета на основном сервере, если реплика не доступна

Нагрузочное тестирование. Настройка

enable_temp_memory_catalog = on

Выносит хранение метаданных ВТ
в оперативную память сеанса

enable_temp_table_on_replica = on

Разрешает создание временных таблиц
и индексов в режиме Read Only

hot_standby_feedback = on
max_standby_streaming_delay = 300s

Разрешение потенциальных конфликтов
при применении WAL на реплике:
40001: ERROR: canceling statement due to conflict with recovery

Нагрузочное тестирование. Документооборот. 1000 пользователей

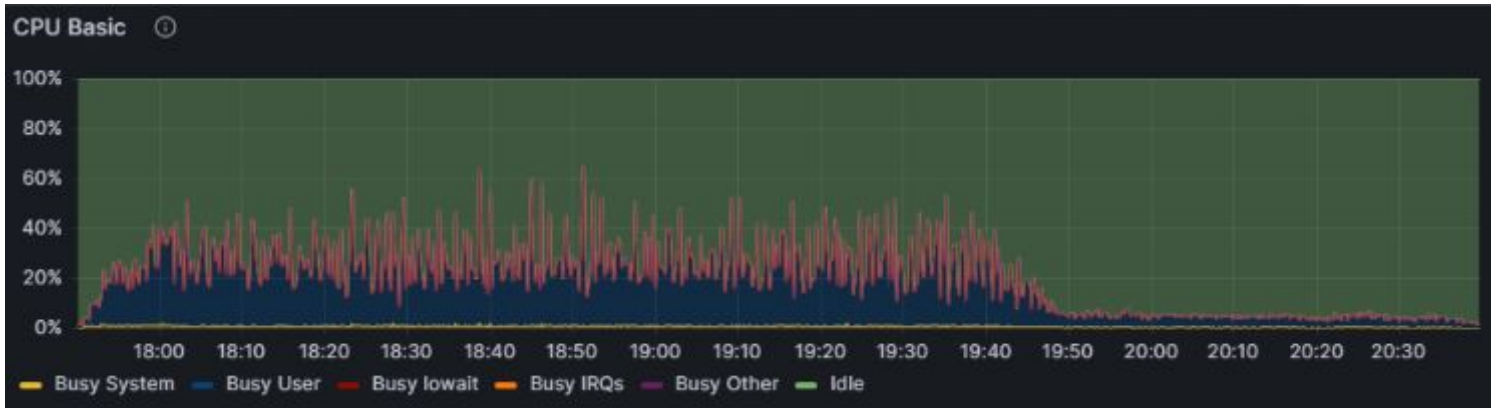
Использование копий баз данных **выключено**

Мастер:

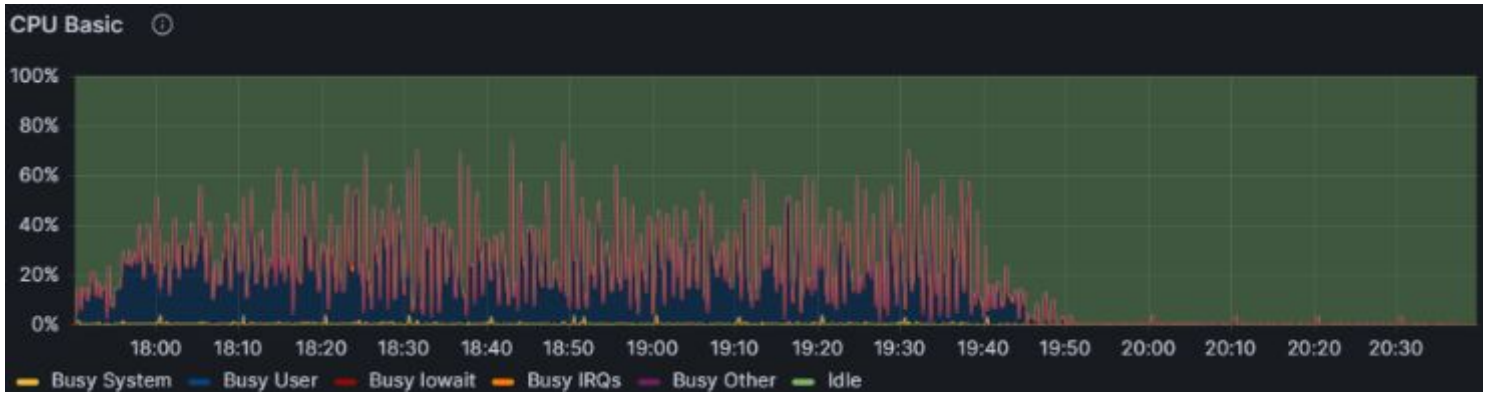
Нагрузочное тестирование. Документооборот. 1000 пользователей

Использование копий баз данных **включено**

Мастер:



Реплика:



Нагрузочное тестирование. ERP. 2000 пользователей

Использование копий баз данных **выключено**

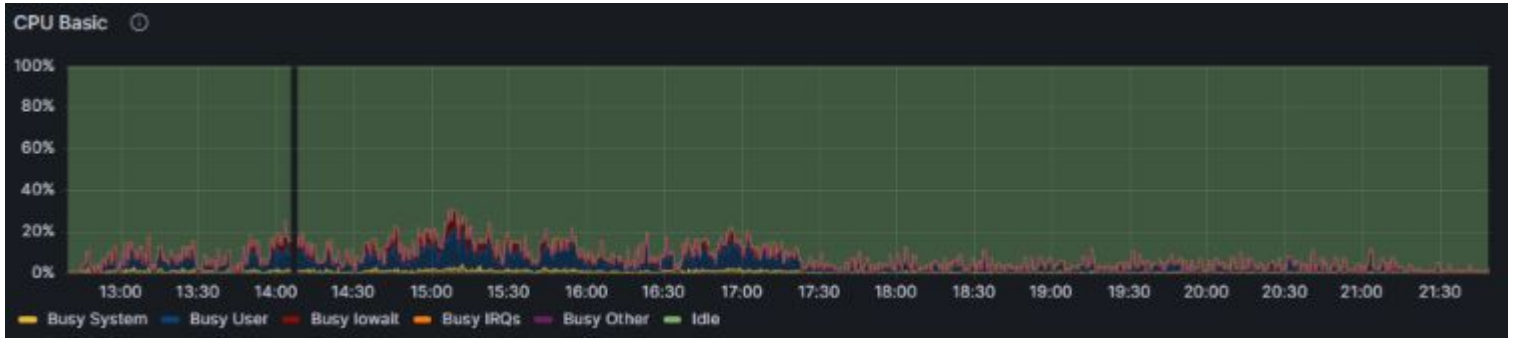
Мастер:

Нагрузочное тестирование. ERP. 2000 пользователей

Использование копий баз данных **включено**

Мастер:

Реплика:



Подробнее — в статье:

tantor

PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

Ваши вопросы?

www.tantorlabs.ru